

■ PLAYWRIGHT

Complete Study Guide

End-to-End Testing | Web Automation | Test Runners

Chapters: Introduction · Architecture · Installation · Fixtures
Locators · XPath · Test Runners · Browser Controls · Annotations

■ Beginner Friendly

■ Hands-on Examples

■ XPath Deep-Dive

■ Best Practices

■ Table of Contents

Chapter 1 — Introduction to Playwright

- › What is Playwright?
- › Key Features & Benefits
- › Architecture Overview

Chapter 2 — Installation & Project Setup

- › Prerequisites
- › Installing Playwright
- › Default Folder Structure

Chapter 3 — In-built Test Runner & Official Docs

- › Playwright Test Runner
- › Running Tests
- › Official Documentation Walkthrough

Chapter 4 — Fixtures: page, browser, context

- › What are Fixtures?
- › Built-in Fixtures
- › Custom Fixtures

Chapter 5 — Hands-on: First Script

- › Writing Your First Test
- › Running & Debugging
- › Common Patterns

Chapter 6 — Test Annotations & Execution Commands

- › Test Annotations
- › Skip / Focus / Tag
- › CLI Commands

Chapter 7 — Browser Controls

- › Navigation Methods
- › Interaction Methods
- › Screenshots & Videos

Chapter 8 — Understanding Web Elements & Locators

- › Locator API

- › CSS Selectors
- › Basic XPath
- › Advanced XPath

Introduction to Playwright

What is Playwright?

Playwright is a modern, open-source end-to-end (E2E) testing framework developed by **Microsoft**. It enables reliable, fast, and powerful automation for web browsers. Unlike older tools such as Selenium, Playwright was designed from the ground up for modern web applications — supporting single-page apps, dynamic content, network interception, and much more.

Developed by	Microsoft (open-source, MIT License)
First Release	January 2020
Language	JavaScript/TypeScript, Python, Java, C#
Browsers	Chromium, Firefox, WebKit (Safari-engine)
GitHub Stars	60,000+ (as of 2024)
Package Name	@playwright/test (Node.js)

Key Features & Benefits

- **Cross-browser:** Single API for Chromium, Firefox, and WebKit.
- **Auto-wait:** Playwright automatically waits for elements to be ready before acting.
- **Network Interception:** Mock APIs and intercept HTTP requests with ease.
- **Multiple Contexts:** Simulate multiple users in the same test.
- **Headless & Headed:** Run tests visually or silently in CI/CD pipelines.
- **Codegen:** Auto-generate test scripts by recording browser interactions.
- **Trace Viewer:** Rich debugging tool that shows screenshots, DOM snapshots & network logs.
- **Parallelism:** Tests run in parallel by default for fast execution.
- **Mobile Emulation:** Simulate mobile devices including touch and geolocation.

Architecture Overview

Playwright's architecture is built around a **client-server model**. The test code (client) communicates with a long-running browser process (server) via the **DevTools Protocol (CDP)** for Chromium and a custom WebSocket protocol for Firefox and WebKit.

Layer	Component	Description
Test Code	JavaScript/TS/Python	Your test scripts using Playwright API
Playwright Node.js	playwright-core	Core library; browser management
Browser Driver	CDP / WebSocket	Low-level communication protocol
Browser	Chromium / FF / WebKit	Actual browser executing actions

■ *Playwright spins up real browsers — no simulation. This means your tests run in the exact same environment as your end users.*

Installation & Project Setup

Prerequisites

- **Node.js** v16 or higher — Download from nodejs.org
- **npm** or **yarn** package manager (bundled with Node.js)
- A code editor — **VS Code** is highly recommended
- Basic knowledge of JavaScript/TypeScript

Installing Playwright

Run the following command in your terminal to scaffold a new Playwright project:

```
# Create a new Playwright project
npm init playwright@latest

# OR install into an existing project
npm install -D @playwright/test

# Install browsers (Chromium, Firefox, WebKit)
npx playwright install

# Install specific browser only
npx playwright install chromium
```

During `{c('npm init playwright@latest')}`, the CLI will ask you:

- Do you want to use TypeScript or JavaScript?
- Where to put your end-to-end tests? (default: tests/)
- Add a GitHub Actions workflow? (for CI/CD)
- Install Playwright browsers? (recommended: Yes)

Default Folder Structure

After initialisation, your project will have this structure:

```
my-playwright-project/
■■■ node_modules/ # Installed dependencies
■■■ tests/ # Your test files go here
■ ■■■ example.spec.ts # Sample test file
■■■ tests-examples/ # More example tests
■ ■■■ demo-todo-app.spec.ts
■■■ playwright.config.ts # Main configuration file
■■■ package.json # Node project manifest
■■■ package-lock.json # Dependency lock file
```

playwright.config.ts — Key Settings

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests', // Where tests live
  fullyParallel: true, // Run tests in parallel
  retries: 2, // Retry failed tests
  workers: 4, // Parallel workers
  reporter: 'html', // HTML report
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry', // Capture trace on retry
    screenshot: 'only-on-failure',
    headless: true, // Run without UI
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

■ The 'projects' array lets you run the same tests across multiple browsers automatically — one of Playwright's most powerful features!

In-built Test Runner & Official Docs

Playwright Test Runner

Playwright ships with its own built-in test runner called **@playwright/test**. You do NOT need Jest, Mocha, or any other testing framework. The runner provides everything out of the box: parallel execution, fixtures, assertions, retries, and rich reporters.

- **test()** — Define a test case
- **expect()** — Built-in assertion library (based on Jest's expect)
- **describe()** — Group related tests
- **beforeAll / afterAll** — Suite-level setup and teardown
- **beforeEach / afterEach** — Per-test setup and teardown

Running Tests — CLI Commands

```
# Run all tests
npx playwright test

# Run a specific file
npx playwright test tests/login.spec.ts

# Run in headed (visible browser) mode
npx playwright test --headed

# Run with UI mode (interactive)
npx playwright test --ui

# Run specific browser only
npx playwright test --project=chromium

# View HTML report
npx playwright show-report

# Generate code (record interactions)
npx playwright codegen https://example.com
```

Official Documentation Walkthrough

Main Docs	playwright.dev/docs/intro
API Reference	playwright.dev/docs/api/class-playwright
Codegen Guide	playwright.dev/docs/codegen
Trace Viewer	playwright.dev/docs/trace-viewer
GitHub Repo	github.com/microsoft/playwright
VS Code Plugin	Search 'Playwright Test' in VS Code extensions

■ *The VS Code extension for Playwright lets you run, debug, and record tests directly inside your editor — highly recommended for beginners!*

Fixtures: page, browser, context

What are Fixtures?

Fixtures are the **backbone of Playwright tests**. They set up the environment before each test and clean up after it. Playwright's test runner provides several built-in fixtures. You can also create your own custom fixtures.

Built-in Fixtures

page	A fresh Page object for each test. Most commonly used fixture.
browser	Shared Browser instance. Use to create custom contexts/pages.
context	A BrowserContext — like an incognito window. Isolated from others.
browserName	Name of the current browser: 'chromium', 'firefox', or 'webkit'.
request	API request context for making HTTP requests in tests.

page fixture

The most commonly used fixture. Each test gets a brand-new page automatically.

```
import { test, expect } from '@playwright/test';

test('page fixture example', async ({ page }) => {
  await page.goto('https://playwright.dev');
  await expect(page).toHaveTitle(/Playwright/);
});
```

browser fixture

Access the raw Browser object to create multiple contexts or pages manually.

```
test('browser fixture', async ({ browser }) => {
  const context = await browser.newContext();
  const page = await context.newPage();
  await page.goto('https://example.com');
  await context.close();
});
```

context fixture

BrowserContext is like an isolated browser session (similar to incognito).

```
test('context fixture', async ({ context }) => {
  const page1 = await context.newPage();
  const page2 = await context.newPage();
  await page1.goto('https://example.com');
  await page2.goto('https://playwright.dev');
  // Both pages share cookies within this context
});
```

Custom Fixtures

Extend the base fixtures to create reusable setup logic:

```
import { test as base } from '@playwright/test';

// Define a custom fixture
const test = base.extend<{ loggedInPage: any }>({
  loggedInPage: async ({ page }, use) => {
    // Setup: login before test
    await page.goto('/login');
    await page.fill('#email', 'user@example.com');
    await page.fill('#password', 'secret');
    await page.click('button[type=submit]');
    await page.waitForURL('/dashboard');

    // Provide the fixture value
    await use(page);

    // Teardown: logout after test
    await page.click('#logout');
  },
});

test('dashboard test', async ({ loggedInPage }) => {
  await expect(loggedInPage).toHaveURL('/dashboard');
});
```

■ Custom fixtures eliminate code duplication — define login once, reuse everywhere!

Hands-on: First Script

Writing Your First Test

Let's write a complete test that navigates to a website, fills a form, and verifies the result.

```
// tests/first-test.spec.ts
import { test, expect } from '@playwright/test';

test.describe('My First Playwright Tests', () => {

  test('should navigate and check title', async ({ page }) => {
    // 1. Navigate to URL
    await page.goto('https://playwright.dev/');

    // 2. Assert the page title
    await expect(page).toHaveTitle(/Playwright/);

    // 3. Click a link
    await page.getByRole('link', { name: 'Get started' }).click();

    // 4. Verify navigation happened
    await expect(page).toHaveURL(/.*intro/);
  });

  test('should fill a search form', async ({ page }) => {
    await page.goto('https://www.google.com');

    // Find the search input and type
    await page.getByRole('combobox', { name: 'Search' }).fill('Playwright
    testing');

    // Press Enter
    await page.keyboard.press('Enter');

    // Wait for results and verify
    await expect(page.getByRole('heading', { name: /playwright/i })).toBeVisible();
  });
});
```

Common Assertions

toHaveTitle(/regex/)	Assert the page title matches a pattern
toHaveURL(/regex/)	Assert the current URL matches
toBeVisible()	Assert element is visible on page
toBeHidden()	Assert element is not visible
toHaveText('...')	Assert element contains specific text
toHaveValue('...')	Assert input field has specific value
toBeChecked()	Assert checkbox/radio is checked
toBeEnabled()	Assert element is not disabled
toHaveCount(n)	Assert number of matching elements
toHaveAttribute(k, v)	Assert element has specific attribute

Setup & Teardown Hooks

```
test.describe('Suite with hooks', () => {

  test.beforeAll(async ({ browser }) => {
    // Runs ONCE before all tests in this suite
    console.log('Starting test suite...');
  });

  test.afterAll(async () => {
    // Runs ONCE after all tests complete
    console.log('Suite done!');
  });

  test.beforeEach(async ({ page }) => {
    // Runs before EACH test – great for login
    await page.goto('https://myapp.com/login');
  });

  test.afterEach(async ({ page }) => {
    // Runs after EACH test – cleanup
    await page.close();
  });

  test('my test', async ({ page }) => {
    // page is already at /login from beforeEach
  });
});
```

Test Annotations & Execution Commands

Test Annotations

Annotations allow you to control test execution without removing test code. They are declarative markers placed before or within tests.

[illegible]

Test Execution Commands — Complete Reference

npx playwright test	Run all tests
npx playwright test --headed	Run with visible browser
npx playwright test --ui	Launch interactive UI mode
npx playwright test --debug	Launch step-by-step debugger
npx playwright test login.spec.ts	Run specific file

npx playwright test -g 'login'	Run tests matching name
npx playwright test --grep @smoke	Run tests with @smoke tag
npx playwright test --project=firefox	Run only Firefox tests
npx playwright test --workers=4	Set parallel workers
npx playwright test --retries=2	Retry failed tests 2 times
npx playwright test --timeout=60000	Set timeout (ms)
npx playwright test --reporter=list	Use list reporter
npx playwright show-report	Open HTML report in browser
npx playwright codegen URL	Record test script
npx playwright trace view trace.zip	View a trace file

■ Use '--ui' mode during development — it's interactive, shows real-time test results, and lets you time-travel through test steps!

Browser Controls: Methods & Screenshots

Navigation Methods

```
// Navigate to a URL
await page.goto('https://example.com');
await page.goto('https://example.com', { waitUntil: 'networkidle' });

// Browser history
await page.goBack();
await page.goForward();
await page.reload();

// Get current URL and title
const url = page.url();
const title = await page.title();

// Wait for navigation to complete
await page.waitForURL('**/dashboard');
await page.waitForLoadState('networkidle');
```

Interaction Methods

```
// Click elements
await page.click('#submit-btn');
await page.dblclick('.item');
await page.click('#btn', { button: 'right' }); // right-click

// Fill inputs
await page.fill('#email', 'user@test.com');
await page.type('#search', 'query', { delay: 100 }); // human-like typing
await page.clear('#input'); // clear input

// Keyboard actions
await page.keyboard.press('Enter');
await page.keyboard.press('Control+A');
await page.keyboard.type('Hello World');

// Mouse actions
await page.mouse.move(100, 200);
await page.mouse.click(300, 400);
await page.hover('#menu-item');

// Dropdowns / Select
await page.selectOption('#country', 'IN');
await page.selectOption('#multi', ['opt1', 'opt2']);

// Checkbox and Radio
await page.check('#agree');
await page.uncheck('#newsletter');

// File upload
await page.setInputFiles('#upload', 'path/to/file.pdf');

// Scroll
await page.evaluate(() => window.scrollTo(0, 500));
await locator.scrollIntoViewIfNeeded();
```

Screenshots & Videos

```
// Full page screenshot
await page.screenshot({ path: 'screenshot.png', fullPage: true });

// Screenshot of specific element
const el = page.locator('.hero-banner');
await el.screenshot({ path: 'banner.png' });

// Capture video (set in config)
// playwright.config.ts:
// use: { video: 'on' } // always record
// use: { video: 'retain-on-failure' }

// Trace recording (set in config)
// use: { trace: 'on-first-retry' }

// Programmatic trace
await context.tracing.start({ screenshots: true, snapshots: true });
// ... run test steps ...
await context.tracing.stop({ path: 'trace.zip' });

// View trace
// npx playwright show-trace trace.zip
```

■ *Traces are incredibly powerful for debugging CI failures. They capture DOM snapshots, network requests, console logs, and screenshots together.*

Understanding Web Elements & Locators

The Locator API

Playwright's **Locator API** is the recommended way to find elements. Unlike old-style selectors, locators are **lazy** (they don't immediately search the DOM), **auto-waiting** (they wait for the element to appear), and **auto-retrying** (they retry on failure). This makes tests far more stable.

```
// Role-based locators (most recommended – accessible)
page.getByRole('button', { name: 'Submit' })
page.getByRole('link', { name: 'Home' })
page.getByRole('textbox', { name: 'Email' })
page.getByRole('heading', { name: 'Dashboard' })

// Label-based (great for forms)
page.getByLabel('Password')

// Placeholder text
page.getByPlaceholder('Enter your email')

// Text content
page.getByText('Welcome back!')
page.getByText(/welcome/i) // regex

// Alt text (images)
page.getByAltText('Company Logo')

// Title attribute
page.getByTitle('Close modal')

// Test ID (data-testid attribute – recommended for stable tests)
page.getByTestId('login-button')

// Generic locator (CSS, XPath, text)
page.locator('#submit')
page.locator('.nav-link')
page.locator('//button[@type="submit"]')
```

CSS Selectors

#id	Select element by ID: #loginBtn
-----	---------------------------------

.class	Select by class: .nav-item
tag	Select by HTML tag: button, input
tag.class	Tag + class: div.container
[attr]	Has attribute: [required]
[attr='val']	Attribute equals value: [type='submit']
[attr*='val']	Attribute contains: [class*='btn']
[attr^='val']	Attribute starts with: [href^='https']
parent > child	Direct child: ul > li
ancestor descendant	Any descendant: form input
:nth-child(n)	Nth child: li:nth-child(2)
:first-child	First child element
:last-child	Last child element
:not(selector)	Negation: button:not([disabled])
a, b	Multiple selectors: button, a

Basic XPath

XPath (XML Path Language) is a query language for navigating HTML/XML. Playwright supports both CSS and XPath in `{c('page.locator')}`.

```
// Absolute path (fragile – avoid)
//html/body/div/form/input

// Relative path (recommended)
//input[@id='email']
//button[@type='submit']
//div[@class='container']

// XPath by text content
//button[text()='Login']
//a[contains(text(),'Sign up')]

// XPath with partial attribute
//input[contains(@class,'search')]
//div[starts-with(@id,'item-')]

// Parent / sibling axis
//label[text()='Email']/following-sibling::input
//input[@id='user']/parent::div

// Index (1-based in XPath!)
(//li[@class='item'])[1] // first item
(//li[@class='item'])[last()]

// In Playwright:
await page.locator('//button[@type="submit"]').click();
```

Advanced XPath

Advanced XPath expressions allow you to traverse the DOM in complex ways:

[illegible]

XPath vs CSS — When to Use Which?

CSS — Use when	Element has stable ID, class, or attribute. CSS is faster and more readable.
XPath — Use when	Need to navigate UP the DOM (parent/ancestor), match by text, or handle complex relationships.

Role — Best for	Accessible elements: buttons, links, inputs. Matches how screen readers see the page.
TestId — Best for	Most stable option. Ask devs to add data-testid attributes to key elements.

■ Playwright recommends: *getByRole* > *getByLabel* > *getByText* > *getByTestId* > *CSS* > *XPath*. Start with the most semantic locator and fall back to *CSS/XPath* only when necessary.